

Omega TK Chatbot

Known Issues & Remediation Plan | AAI 6670 Conversational AI | Northeastern University

Eight confirmed issues were identified in the production codebase through direct code inspection. All eight are real. This document describes each problem in plain terms, explains the practical impact on users and system reliability, and maps every issue to a concrete resolution organized across three independently deployable sprints. No changes require a full rewrite. Each sprint can ship on its own.

Issue Summary

Issue	Description	Priority	Sprint
1	Knowledge base resets on every new chat session	P1 Critical	Sprint 3
2	Feedback table cannot join to analytics log	P1 Critical	Sprint 1
3	Conversation summary cache wiped on server restart	P2 High	Sprint 2
4	Knowledge retrieval loads all chunks into RAM	P2 High	Sprint 3
5	RAG evaluation is only one-third complete	P3 Medium	Sprint 1
6	Thumbs-down feedback never closes the loop	P3 Medium	Sprint 2
7	No user authentication — sessions are anonymous	P3 Medium	Sprint 3
8	Adversarial test coverage is thin	P3 Medium	Sprint 1

Database Schema Changes (run first in Supabase SQL Editor)

All three sprints depend on these schema additions. Run them once before any code changes are deployed. They are additive and non-breaking.

```
ALTER TABLE queries_log ADD COLUMN message_id TEXT;
ALTER TABLE queries_log ADD COLUMN answer_relevance_score FLOAT;
ALTER TABLE knowledge_chunks ADD COLUMN user_id TEXT;
CREATE INDEX IF NOT EXISTS idx_knowledge_chunks_user_id
ON knowledge_chunks(user_id);

CREATE TABLE summaries (
  hash TEXT PRIMARY KEY,
  summary TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Sprint 1 — Backend Hardening + Tests

Pure backend and test changes. No UI work. Independently deployable in a single PR. Adds 11 adversarial tests, closes the analytics join gap, and begins tracking answer relevance on every query.

8

Adversarial test coverage is thin

Sprint 1

What's broken	The entire adversarial test suite contains exactly two tests: one for 'ignore your instructions' and one for 'pretend you are a different AI.' The blocklist has more words in it than it has tests. Injection variants, roleplay bypasses, and context manipulation patterns are completely untested.
Why it matters	A gap in the blocklist would only be discovered when a real user finds it. Untested defenses are unverified defenses. The test taxonomy for production LLM systems calls for 10 to 20 percent adversarial coverage. The current suite sits well below that.
Proposed fix	Add five new blocklist terms to <code>src/guardrails.py</code> : 'no restrictions', 'without restrictions', 'developer mode', 'do anything now', and 'unrestricted'. Add a new <code>TestL1Adversarial</code> class to <code>tests/test_guardrails.py</code> with 11 test cases covering prompt injection, roleplay bypass, context manipulation, and DAN-style jailbreak patterns. Target: 70 tests passing, up from 59.
Files touched	<code>src/guardrails.py</code> <code>tests/test_guardrails.py</code>

2

Feedback table cannot join to analytics log

Sprint 1

What's broken	The feedback table stores a <code>message_id</code> alongside each rating, but <code>queries_log</code> never saves a <code>message_id</code> . The two tables cannot be joined. There is no way to answer: which specific responses are users rating badly, and what did the FAISS score and retry count look like for those responses?
Why it matters	Feedback is collected but is not actionable. The helpful rate shown on the analytics dashboard is an aggregate percentage with no path to the underlying responses. Debugging quality regressions requires guesswork instead of data.
Proposed fix	Generate a UUID <code>message_id</code> at the start of each request. Save it to <code>queries_log</code> and return it in the API response. The frontend stores <code>message_id</code> on each bot message object and sends it when a user submits thumbs feedback. One new column, roughly ten lines of code in <code>app.py</code> .
Files touched	<code>server/app.py</code> <code>client/src/App.jsx</code>

What's broken queries_log captures faiss_score, which measures retrieval confidence. The other two dimensions of the RAG Triad, answer relevance and faithfulness, are not tracked. The system has no way to detect whether responses are drifting off-topic or whether GPT is adding information from outside the retrieved documentation.

Why it matters Hallucinations that slip past Layer 3 validation and responses that technically answer a slightly different question will never appear in analytics. Quality degradation over time becomes invisible.

Proposed fix After generation, embed both the original query and the response text using the existing _embed_text helper. Compute cosine similarity and log it as answer_relevance_score in queries_log. This uses no new dependencies and adds one embedding call per non-fallback response. About 20 lines in app.py.

Files touched server/app.py

Sprint 2 — Analytics Loop + Summary Persistence

Closes the feedback loop so thumbs-down ratings surface actionable data, and persists the conversation summary cache to survive server restarts. Depends on Sprint 1 being deployed first.

6	Thumbs-down feedback never closes the loop	Sprint 2
What's broken	Feedback is written to Supabase and a helpful rate percentage is shown in the dashboard. Nothing downstream happens. No response is flagged, no alert fires, and the feedback data has never been used to adjust a prompt or identify a pattern of failure.	
Why it matters	Collecting feedback with no downstream action is measuring for the sake of measuring. If twenty users rate the same type of response badly, the system has no mechanism to surface that pattern. The evaluation loop described in the course material exists in name only.	
Proposed fix	In <code>analytics_endpoint</code> , add a query that fetches the five most recent thumbs-down ratings and joins them to <code>queries_log</code> by <code>message_id</code> to retrieve the original message, intent, and latency. Return this as <code>recent_negative</code> in the analytics response. In <code>AnalyticsDashboard.jsx</code> , render a 'Needs Review' card showing each low-rated message when data is present.	
Files touched	<code>server/app.py</code> <code>client/src/components/AnalyticsDashboard.jsx</code>	

3	Conversation summary cache wiped on server restart	Sprint 2
What's broken	The system summarizes older conversation turns (beyond the six-message sliding window) and caches those summaries in a Python dict keyed by an MD5 hash. This dict lives entirely in RAM. Every server restart wipes it. Full conversation history is safe in Supabase. Only the summary cache is lost.	
Why it matters	When a user has a long conversation and the server restarts mid-session, the next message triggers a fresh re-summarization of the older turns. This costs extra OpenAI tokens and adds latency to that one request. The user sees no error, which makes the degradation silent and hard to diagnose.	
Proposed fix	Upgrade <code>_get_summary()</code> to a three-tier cache: L1 is the existing in-memory dict (instant), L2 is the Supabase summaries table (read on miss, write on new summary), L3 is the GPT call as a last resort. On a cache miss, the function checks Supabase before calling GPT. Summaries now survive restarts with no change to the API contract or user-facing behavior.	
Files touched	<code>server/app.py</code> (Supabase summaries table required from schema step)	

Sprint 3 — Knowledge Base Architecture

The largest sprint. Introduces a persistent browser user identity so knowledge uploads survive New Chat and work across devices. Applies the RAM safety cap to knowledge retrieval as a near-term scalability guard.

1	Knowledge base resets on every new chat session	Sprint 3
What's broken	Every knowledge chunk is tagged with a session_id generated by the browser and stored in localStorage. Clicking New Chat creates a new session_id. The old knowledge chunks still exist in Supabase but the lookup key changed, so the system cannot find them. The data is not deleted. It is just permanently invisible to the new session.	
Why it matters	A researcher who uploads custom OEMacrocycle notes loses all of that context the moment they start a fresh conversation. This makes the Knowledge Base feature unreliable enough that users will stop using it after experiencing the reset once.	
Proposed fix	Introduce a second localStorage key, omega_user_id, that is generated once and never reset, even when New Chat is clicked. Tag knowledge chunks with user_id instead of session_id. At query time, filter by user_id when present, falling back to session_id for existing chunks. Knowledge now follows the user across sessions and devices without requiring Supabase Auth.	
Files touched	client/src/App.jsx client/src/api.js client/src/components/KnowledgePanel.jsx client/src/components/MiddlePanel.jsx server/app.py	

4	Knowledge retrieval loads all chunks into RAM	Sprint 3
What's broken	When a query arrives and the user has uploaded knowledge, the system fetches every single chunk for the session from Supabase into Python memory, including full 1536-dimension embedding arrays, then runs cosine similarity in a Python loop to find the top three matches. This is confirmed in lines 201 to 207 of app.py.	
Why it matters	Fine with five uploads. Breaks with fifty. Latency spikes as the knowledge base grows and the server holds all embedding arrays simultaneously. There is no graceful degradation and the failure is invisible until it happens.	
Proposed fix	Apply a 500-row safety cap to the Supabase query as an immediate guard. Add a log warning when the chunk count exceeds 100. The long-term fix is enabling the pgvector extension in Supabase and moving similarity search to SQL, turning the Python loop into a single ORDER BY vector distance query with LIMIT 3. This is applied in the same function as Issue 1 to avoid touching the code twice.	
Files touched	server/app.py (_retrieve_knowledge function – apply with Issue 1)	

7	No user authentication — sessions are anonymous	Sprint 3
What's broken	Users are identified by a UUID in localStorage. Two devices means two identities. Clearing browser storage means permanent loss of session association. There is no concept of a persistent account and no Supabase Auth anywhere in the codebase.	
Why it matters	Beyond the UX inconvenience, anonymous sessions block the fix for Issue 1 at its most complete form. They also make the system harder to audit and impossible to isolate on a per-user basis if knowledge base isolation becomes important.	
Proposed fix	Sprint 3 introduces omega_user_id as a persistent browser identity, which solves the cross-session knowledge problem without requiring a login flow. Full Supabase Auth with email and password login remains a V3 scope item that should follow once the knowledge base migration is stable. omega_user_id is not real auth. It is a pragmatic stepping stone.	
Files touched	Addressed partially by Issue 1 fix Full auth: V3 scope	

The system is production-deployed and all 59 automated tests pass. These three sprints address every confirmed issue without requiring a rewrite of core pipeline logic. Sprint 1 ships in a single backend PR and raises the test count to 70. Sprint 2 activates the feedback loop the evaluation architecture already has wiring for. Sprint 3 fixes the only issue that currently causes users to lose data they intentionally saved.